

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – макроопределения»

Выполнила:

Студентка 4 курса

Группы АС-66

Прокурат В.Д.

Проверила:

Кулик А.Д.

Цель работы: изучение приёмов разработки макроопределений и использования их в программах.

Вариант 20 (4)

- Постановка задачи.** 1. Изучить состав и средства задания макроопределений.
2. Написать макроопределение, реализующее функцию заданного преподавателем варианта работы.
3. Выполнить проверку работоспособности разработанного макроопределения.

Задание: переместить заданный символ, если он содержится в строке, в начало строки.

Код программы:

```
.386
.model flat, stdcall
option casemap:none

; ---- прототипы WinAPI ----
GetStdHandle     PROTO :DWORD
ReadConsoleA     PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
WriteConsoleA    PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
ExitProcess      PROTO :DWORD

includelib kernel32.lib

STD_INPUT_HANDLE equ -10
STD_OUTPUT_HANDLE equ -11
MAX_LEN          equ 100

;=====
; Макроопределение: переместить символ CH в начало строки STR
; Если символа нет – строка не меняется.
; Параметры:
;   STR – адрес строки (в буфере, оканчивается нулём)
;   CH  – искомый символ (байт)
; Регистры, которые портит: eax, ebx, ecx, edx, esi
;=====
MOVE_CHAR_TO_FRONT MACRO STR, CH
    LOCAL found, done

    lea esi, STR          ; esi = адрес строки
    xor ecx, ecx          ; ecx = индекс

search_loop:
    mov al, [esi + ecx]   ; al = STR[ecx]
    test al, al
    jz  done              ; конец строки – символ не найден
    cmp al, CH            ; это искомый символ?
    je  found             ; да – нашли
    inc ecx
    jmp search_loop

found:
    ; ecx = индекс найденного символа
    test ecx, ecx
    jz  done              ; уже на позиции 0 – ничего делать не надо

    mov al, [esi + ecx]   ; al = найденный символ
shift_loop:
    mov bl, [esi + ecx - 1] ; bl = предыдущий символ
    mov [esi + ecx], bl   ; сдвигаем его вправо
    dec ecx
    jnz shift_loop        ; пока не дошли до позиции 1

    mov [esi], al         ; ставим искомый символ в начало
```

```

done:
ENDM
;=====

.data
    hIn      dd 0
    hOut      dd 0
    nWrite    dd 0
    nRead     dd 0
    buffer    db MAX_LEN + 2 dup(0)

    target    db 'o'                ; символ, который ищем

    sPrompt   db "Type a line (max 100 chars, finish with a dot):", 13, 10
    sPromptLen equ $ - sPrompt

    sTargetInfo db 13, 10, "Target character: 'o'. If found, it moves to the front.", 13, 10
    sTargetLen  equ $ - sTargetInfo

    sBefore    db "Before: "
    sBeforeLen  equ $ - sBefore

    sAfter     db "After: "
    sAfterLen   equ $ - sAfter

    sNewLine   db 13, 10
    sNewLineLen equ $ - sNewLine

    sNoDot     db "The dot was not found at the end. Try again.", 13, 10
    sNoDotLen  equ $ - sNoDot

.code

;-----
; WriteStr: печатает строку (edx = адрес, ecx = длина)
;-----
WriteStr proc uses eax
    push 0
    push offset nWrite
    push ecx
    push edx
    push hOut
    call WriteConsoleA
    ret
WriteStr endp

;-----
; StrLen: длина C-строки (edx = адрес), возвращает в ecx
;-----
StrLen proc uses esi edx
    lea esi, [edx]
    xor ecx, ecx
sl_loop:
    cmp byte ptr [esi + ecx], 0
    je  sl_done
    inc ecx
    jmp sl_loop
sl_done:
    ret
StrLen endp

;-----
; Точка входа
;-----
start:
    push STD_INPUT_HANDLE
    call GetStdHandle
    mov hIn, eax

    push STD_OUTPUT_HANDLE
    call GetStdHandle

```

```

mov hOut, eax

mov edx, offset sPrompt
mov ecx, sPromptLen
call WriteStr

push 0
push offset nRead
push MAX_LEN + 1
push offset buffer
push hIn
call ReadConsoleA

; находим точку и меняем на нуль-терминатор
mov ecx, nRead
lea esi, buffer
xor edx, edx
findDot:
    cmp edx, ecx
    jge noDot
    cmp byte ptr [esi + edx], '.'
    je foundDot
    inc edx
    jmp findDot

foundDot:
    mov byte ptr [esi + edx], 0
    jmp afterDot

noDot:
    mov edx, offset sNoDot
    mov ecx, sNoDotLen
    call WriteStr
    push 0
    call ExitProcess

afterDot:
    ; ---- информация об искомом символе ----
    mov edx, offset sTargetInfo
    mov ecx, sTargetLen
    call WriteStr

    ; ---- печатаем "Before: " + исходная строка ----
    mov edx, offset sBefore
    mov ecx, sBeforeLen
    call WriteStr

    lea edx, buffer
    call StrLen
    lea edx, buffer
    call WriteStr

    mov edx, offset sNewLine
    mov ecx, sNewLineLen
    call WriteStr

    ; ---- вызываем макрос ----
    MOVE_CHAR_TO_FRONT buffer, target

    ; ---- печатаем "After: " + результат ----
    mov edx, offset sAfter
    mov ecx, sAfterLen
    call WriteStr

    lea edx, buffer
    call StrLen
    lea edx, buffer
    call WriteStr

    mov edx, offset sNewLine
    mov ecx, sNewLineLen
    call WriteStr

```

```
push 0  
call ExitProcess
```

end start

```
D:\politech\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab3>lab3.exe  
Type a line (max 100 chars, finish with a dot):  
abcd.
```

```
Target character: 'o'. If found, it moves to the front.  
Before: abcd  
After:  abcd
```

```
D:\politech\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab3>lab3.exe  
Type a line (max 100 chars, finish with a dot):  
open.
```

```
Target character: 'o'. If found, it moves to the front.  
Before: open  
After:  open
```

```
D:\politech\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab3>lab3.exe  
Type a line (max 100 chars, finish with a dot):  
prokurator.
```

```
Target character: 'o'. If found, it moves to the front.  
Before: prokurator  
After:  oprkurator
```

Вывод: в ходе работы изучены макроопределения MASM: их структура (заголовок MACRO, тело, концевик ENDM), способы передачи параметров, директива LOCAL для уникальных меток. Разработано макроопределение MOVE_CHAR_TO_FRONT, которое перемещает заданный символ в начало строки, если он в ней есть. Проверка была выполнена на трёх случаях: символ отсутствует, уже стоит первым или присутствует в середине строки. Использование макроса упрощает вызов, т.к. параметры передаются по имени, без загрузки в регистры, как в процедуре.